

ASSIGNMENT 1

This assignment contains 5 theoretical exercises and 6 programming exercises.

- Solutions to all theoretical exercises must be submitted in a **single** file in PDF format.
- Solutions to all programming exercise must be submitted in a **single** JAVA file, completing the template that you can find in the course moodle website. The file must be self-contained and must work with Java 17 or greater. You cannot use any external Java libraries. If you need them, you are allowed to use `java.util.ArrayList`, `java.util.LinkedList`, `java.util.Stack`, and `java.util.Queue`.

Theoretical exercises

Exercise 1. [15 pts]: Loop invariants

Suppose that A is an array of n integers. We assume as a precondition that $n > 0$. Consider the following algorithm to *rotate* the array A to the left. For example, rotating $[1, 2, 3, 4]$ yields $[2, 3, 4, 1]$.

```
i = n - 1;
x = A[0];
while (i >= 0) {
    y = A[i];
    A[i] = x;
    x = y;
    i = i - 1;
}
```

Let A_{start} denote the array right before the algorithm starts, and let A_{end} denote the array right after the algorithm ends. Use the **loop invariant theorem** to prove that the algorithm rotates the array. Formally, you must prove that `isLeftRotation( $A_{\text{start}}$ ,  $A_{\text{end}}$ )` holds, where in general the following predicate:

$$\text{isLeftRotation}(A, B)$$

means that:

$$\begin{aligned} & |A| = |B| \\ \wedge & B[n-1] = A[0] \\ \wedge & \forall i \in \mathbb{Z}. (0 \leq i < |A| - 1 \implies B[i] = A[i+1]) \end{aligned}$$

Exercise 2. [10 pts]: “O” notation

We write $\mathbb{R}_{\geq 1}$ for the set $\{x \in \mathbb{R} \mid x \geq 1\}$. Prove the following properties:

- Let $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 1}$ and let $k \in \mathbb{N}$. If $f(n), g(n) \in O(h(n))$ then $(f(n) + g(n))^k \in O(h(n)^k)$.
- Let $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 1}$ be a monotonically non-decreasing function¹ such that $f(n) \in O(n)$. Then $\sum_{i=1}^n f(i)^2 \in O(n^3)$.
- Let $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 1}$ such that $f(n) \in O(g(n)^2)$ and $f(n) \in O(h(n)^2)$. Then $f(n) \in O(g(n) \cdot h(n))$.

Exercise 3. [5 pts]: Time complexity

- Write pseudo-code for an algorithm that takes a binary search tree and returns the sum of the minimum value and the maximum value in the tree. You should **only use iteration** (`while` loops), not recursion.
- Worst-case time complexity.** Define a family of binary search trees $W_1, W_2, W_3, \dots$ such that W_n has n nodes and such that the time complexity of your algorithm is $\Theta(n)$ when applied to W_n . Conclude that the worst-case time complexity of your algorithm is $\Theta(n)$.
Note. If your algorithm does not behave with this complexity, you should redesign your algorithm.
- Best-case time complexity.** Define a family of binary search trees $B_1, B_2, B_3, \dots$ such that B_n has n nodes and such that the time complexity of your algorithm is $\Theta(1)$ when applied to B_n . Conclude that the best-case time complexity of your algorithm is $\Theta(1)$.
Note. If your algorithm does not behave with this complexity, you should redesign your algorithm.

¹That is, $f(n) \leq f(n+1)$ for every $n \in \mathbb{N}$.

Exercise 4. [15 pts]: Algorithms on AVL trees

- Propose an algorithm (in pseudo-code) that takes as input two AVL trees A and B , and produces as output a **sorted** array containing those elements that appear both in A and in B . The worst-case time complexity must be $O(n \log n)$ where $n = \max\{|A|, |B|\}$. Justify that your algorithm is correct and that it has the stated worst-case time complexity.
- Propose an algorithm (in pseudo-code) that takes as input two integers $\ell, r \in \mathbb{Z}$ such that $\ell \leq r$, and an AVL tree A containing n integers without duplicates. The output should be the sum of all the integers k such that k appears in A and k is in the range $\ell \leq k \leq r$. The time complexity of the algorithm must be $O(\log n + (r - \ell))$. Justify that your algorithm is correct and that it has the stated worst-case time complexity.
- Suppose that $f : \mathbb{N} \rightarrow \mathbb{N}$ is a strictly increasing function² that can be computed in $O(1)$ worst-case time. Propose a data structure that works as follows:
 - It is initialized with an integer n . The initialization phase must take $O(n)$ worst-case time.
 - It provides an operation that receives an integer j and returns a **pre-image**, this is, an integer i such that $f(i) = j$. This operation must take $O(\log n)$ time. **Precondition:** j must be such that there exists an integer i such that $1 \leq i \leq n$ and $f(i) = j$.

Exercise 5. [5 pts]: Proving properties of trees

We define **ternary trees** over a set X inductively as follows:

- nil is a ternary tree.
 - If T_1, T_2, T_3 are ternary trees and $x \in X$, then $\langle x, T_1, T_2, T_3 \rangle$ is also a ternary tree.
- A perfectly balanced ternary tree of height 1 has only one node (the root). A perfectly balanced ternary tree of height 2 has four nodes (the root, and its three children). Define a function $\text{NB}(h)$ such that $\text{NB}(h)$ denotes the number of nodes of a perfectly balanced ternary tree of height h . Prove it by induction.
 - Let n be the number of nodes in a ternary tree of height h (not necessarily balanced). Prove that $n \in O(3^h)$.

Programming exercises

The goal of the following exercises is to implement a **set of integers** based on AVL trees. Download the files for this assignment from the course campus website. Edit the `Main.java` file. The file contains:

- A `Main` class.
- A `Node` class to represent nodes of an AVL tree. A node contains a **key**, links to the **left and right children**, and an integer representing the **height**. The `Main.java` file already includes a constructor to create an AVL tree with just one node. An AVL tree is represented with a `Node`, which can may be either `null` (an empty tree), or non-`null`.
- A `Set` class to represent sets based on AVL trees.

Exercise 6. [10 pts] Height and representation invariant checking

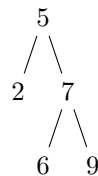
These methods correspond to the `Node` class:

- Define a static method `int slowHeight(Node p)` that takes a node and calculates its height recursively, traversing the whole tree. The worst-case time complexity must be $O(n)$, where n is the number of nodes in the tree.

Note: this method must not be used by the operations of the AVL tree, because it is too slow. But it is useful to have this operation to check that the representation invariant is fulfilled.

²That is, $f(n) < f(n+1)$ for every $n \in \mathbb{N}$.

- b) Define a static method **int fastHeight(Node p)** that takes a node and calculates its height by returning the height stored in the node. The worst-case time complexity must be $O(1)$.
- c) Define a static method **int balanceFactor(Node p)** that returns the balance factor of the node, defined as the difference between the height of the two children. The worst-case time complexity must be $O(1)$.
- d) Define a static method **boolean checkInvariant(Node p)** that returns **true** if and only if the AVL tree fulfills the representation invariant. The following conditions must hold:
- If the node is null, it fulfills the representation invariant.
 - The key of the root must be greater than the key of the left child (if the left child is non-empty).
 - The key of the root must be less than the key of the right child (if the right child is non-empty).
 - The height stored in the node must be the same as the actual height of the node. That is, **slowHeight(p)** and **fastHeight(p)** must return the same integer.
 - The balance factor must be either 1, 0, or -1.
 - The left child must (recursively) fulfill the representation invariant.
 - The right child must (recursively) fulfill the representation invariant.
- e) Complete the static method **void testInvariant()** to construct (manually) the following AVL tree and check that the invariant is fulfilled:

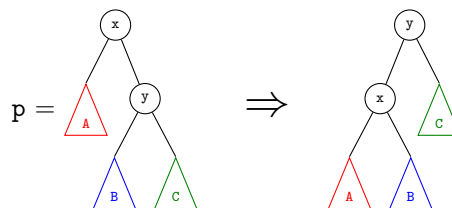


Exercise 7. [10 pts] Rotations

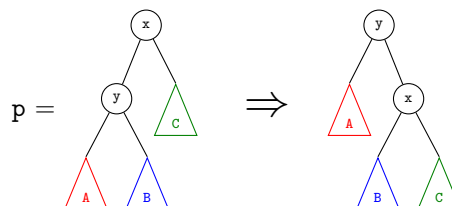
These methods correspond to the **Node** class.

- a) Define a static method **Node rotateLeft(Node p)** that receives a tree shaped like the one on the left and returns a tree shaped like the one on the right.

The method assumes that the children are AVL trees, but the tree at the root is not itself an AVL tree because the balance factor is 2. Moreover, the balance factor of the node **y** must be 0 or 1 at the start, but not -1.

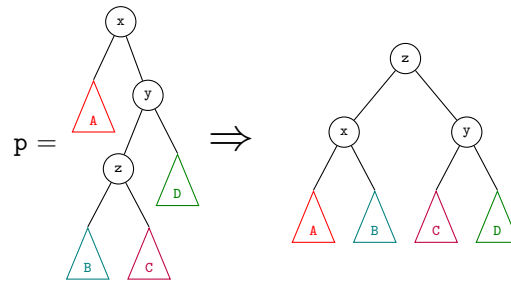


- b) Define a static method **Node rotateRight(Node p)** that performs the symmetric operation:

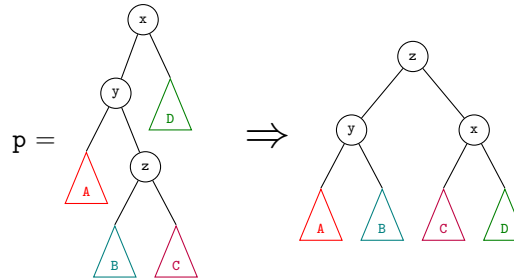


- c) Define a static method **Node doubleRotateLeft(Node p)** that receives a tree shaped like the one on the left and returns a tree shaped like the one on the right.

The method assumes that the children are AVL trees, but the tree at the root is not itself an AVL tree because the balance factor is 2. Moreover, the balance factor of the node **y** must be -1 at the start. Remember that to implement this operation you can do first a right rotation on the right child, and then a left rotation on the root.



d) Define a static method `Node doubleRotateRight(Node p)` that performs the symmetric operation:



The following methods correspond to the `Set` class. The set class is represented using an AVL tree.

Exercise 8. [5 pts] Lookup

Complete the implementation of the method `boolean contains(int x)` that returns `true` if and only if the set contains the given element. The worst-case time complexity should be $O(\log n)$, where n represents the cardinality of the set. You should implement this method iteratively, **without recursion**.

Exercise 9. [10 pts] Insertion

Complete the implementation of the method `void add(int x)` that adds an element to the set. If the element is already present, this operation has no effect. The worst-case time complexity should be $O(\log n)$, where n represents the cardinality of the set. You should implement this method iteratively; you can use auxiliary private methods if you need, but **you must not use recursion**.

Exercise 10. [10 pts] Removal

Complete the implementation of the method `void remove(int x)` that removes an element from the set. If the element is not present, this operation has no effect. The worst-case time complexity should be $O(\log n)$, where n represents the cardinality of the set. You should implement this method iteratively; you can use auxiliary private methods if you need, but **you must not use recursion**.

Exercise 11. [5 pts] Testing

These methods correspond to the `SetTest` class. The `SetTest` already contains a very basic test `test0()`. Add tests to validate at least the following functionality:

- Insert the numbers $1, 2, 3, \dots, 100$. After each insertion, check that all the inserted elements are in the set, that the set does not contain other elements, and that the invariant is preserved.
- Insert the numbers $100, 99, 98, \dots, 1$. After each insertion, check that all the inserted elements are in the set, that the set does not contain other elements, and that the invariant is preserved.
- Find a sequence insertions that require a double left rotation, and check that the invariant is fulfilled afterwards.
- Find a sequence insertions that require a double right rotation, and check that the invariant is fulfilled afterwards.
- Start from an array with the numbers from 1 to 100 and remove them in the following order: $1, 2, 3, \dots, 100$. After each removal, check that all the corresponding elements are in the set, and that the invariant is preserved.
- Start from an array with the numbers from 1 to 100 and remove them in the following order: $100, 99, 98, \dots, 1$. After each removal, check that all the corresponding elements are in the set, and that the invariant is preserved.

Submission

- You must submit your solution using the moodle website.
- Your solution must be submitted as a single JAVA file.
- The deadline for submission is **Thursday 2026-04-30 until 23:59:59**, Shantou time (GMT+8).
- **Late submissions** will be accepted until Friday 2026-05-01 23:59:59 with **25% penalty**.
- No submissions will be accepted after Friday 2026-05-01 23:59:59.
- Sometimes the course moodle experiences delays. Please **do not wait to submit until the last minute**.